

Linear Regression

- Estimates a linear relation between X and Y .

Hypothesis : $Y = h_{\theta}(X) = \theta^T \cdot X$
function

Simplified : $y = \underset{\substack{\uparrow \\ \text{Bias}}}{\theta_0} + \underset{\substack{\uparrow \\ \text{feature} \\ \text{weight}}}{\theta_1} x$

Cost function, $J(\theta) = \text{Mean Squared Error (MSE)}$

$$= \frac{1}{2m} \sum (\hat{y}_i - y_i)^2$$

$\uparrow$
Added for simpler derivative

Simple gradient descent is used for minimisation of $J(\theta)$

$$\nabla J(\theta) = \frac{d}{d\theta} J(\theta) = \frac{1}{m} \sum (\hat{y}_i - y_i) \cdot x_i$$

Simplified code loop :-

```
func gradient_descent( $\theta$ ,  $x$ ,  $y$ , learn_rate):
```

```
    grad = (sum( $\theta \cdot x - y$ )) /  $m$ 
```

```
    return  $\theta - \text{learn\_rate} \cdot \text{grad}$ 
```

```
func linear_regression( $x$ ,  $y$ ):
```

```
    # finds & returns the predicted value of  $\theta$ 
```

```
     $\theta = [0, 0]$ 
```

```
    epochs = 1000
```

```
     $m = \text{len}(x)$ 
```

```
    for epoch in range(epochs):
```

```
         $\theta = \text{gradient\_descent}(\theta, x, y, 0.01)$ 
```

```
    return  $\theta$ 
```

Polynomial Regression:

A modification of linear regression that uses higher exponents of each feature to find a better fitting model.

Uses a similar approach as linear Regression, where $x_0, x_0^2, x_0^3 \dots x_0^n$ become $x_0, x_1, x_2 \dots x_n$.

Pros:-

1. **High Interpretability:** Each weight directly affects the model. Also every weight is much easier to understand and visualize.
2. **Computationally Efficient:** Requires very low processing power. Easily run on non-dedicated hardware for even enormous datasets.
3. **Low chances of Overfitting:** Bcz of inherent simplicity of linear regression, it is unlikely to overfit unless using high degree polynomials.

Cons:-

1. **Likely to Underfit**: Very likely to underfit on complex data.
2. **High sensitivity to Outliers**: Outliers tend to affect the model aggressively requiring manual handling often.
3. **Vulnerable to multicollinearity**: Highly related inputs make weight estimates unstable, erratic.
4. **Limited to Numerical inputs and outputs**:
For more complex applications like classification, need to be modified to other algos like logistic Regression.

Regularized Regression:

1. Ridge Regression (ℓ_2):

$$y = h(x) = \theta_0 + \theta_1 x \leftarrow \text{Unchanged}$$

$$J(\theta) = \text{MSE}(\theta) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2 \leftarrow \theta_0, \text{Bias not included}$$

$$\nabla J(\theta) = \frac{1}{m} \sum_{i=0}^n (\hat{y}_i - y_i) \cdot x_i + \frac{2\alpha}{m} \sum_{i=1}^n \theta_i$$

α = Hyper parameter, rate of regularization

2. Lasso Regression (ℓ_1):

$$y = h(x) = \theta_0 + \theta_1 x$$

$$J(\theta) = \text{MSE}(\theta) + 2\alpha \sum |\theta_i|$$

$$\nabla J(\theta) = \frac{1}{m} \sum_{i=0}^n (\hat{y}_i - y_i) x_i + 2\alpha \sum_{i=1}^n \text{sign}(\theta_i)$$

* High likelihood of getting rid of less effective features. Turns model sparse.

3. Elastic Net Regression (l_1 and l_2):-

$$y = h(x) = \theta_0 + \theta_1 x$$

$$J(\theta) = \text{MSE}(\theta) + \lambda \left(2\alpha \sum_{i=1}^n |\theta_i| \right) + (1-\lambda) \left(\frac{\alpha}{n} \sum_{i=1}^n \theta_i^2 \right)$$

Preferred over the other models above as it shows a mix of the properties of both Ridge and lasso Regression.

